EXPERIMENT -7

AIM- To identify functional dependencies in a denormalized Sales table and normalize the relation step-by-step from UNF to 3NF and BCNF. To verify lossless join and dependency preservation using SQL queries.

OBJECTIVES

1. To identify functional dependencies and candidate keys in a Sales relation.
2. To understand the problems associated with data redundancy and anomalies.
3. To normalize a relation into 1NF, 2NF, 3NF and BCNF.
4. To perform decomposition of relations based on functional dependencies.
5. To verify lossless join and dependency preservation.
6. To reconstruct the original relation using SQL joins.

THEORY

Normalization is a database design technique used to organize data efficiently, minimize redundancy and prevent insertion, deletion and update anomalies.

A denormalized Sales table may contain repeated customer details, product information and multiple products within a single sale record. This can result in inconsistent data and unnecessary duplication.

Functional Dependency

A functional dependency (FD) is a relationship between attributes in which one attribute or a set of attributes uniquely determines another attribute or set of attributes.

It is represented as:

$X \rightarrow Y$

Here, X is called the determinant, and Y is the dependent attribute.

For example:

- SaleID → SaleDate, CustomerID
- ProductID → ProductName, UnitPrice
- (SaleID, ProductID) → Quantity

These dependencies are based on the business rules assumed for this experiment.

Normal Forms

1. Unnormalized Form (UNF)

The relation may contain repeating groups, multi-valued fields or non-atomic values.

Example: A sales record containing multiple product IDs in a single field.

2. First Normal Form (1NF)

A relation is in 1NF when every attribute contains atomic values and there are no repeating groups.

3. Second Normal Form (2NF)

A relation is in 2NF when it is in 1NF and every non-prime attribute is fully dependent on every candidate key, with no partial dependency.

4. Third Normal Form (3NF)

A relation is in 3NF if, for every non-trivial functional dependency $X \rightarrow A$, either:

- X is a superkey, or
- A is a prime attribute.

3NF reduces redundancy caused by problematic dependencies.

5. Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if every determinant of a non-trivial functional dependency is a superkey.

BCNF is stricter than 3NF.

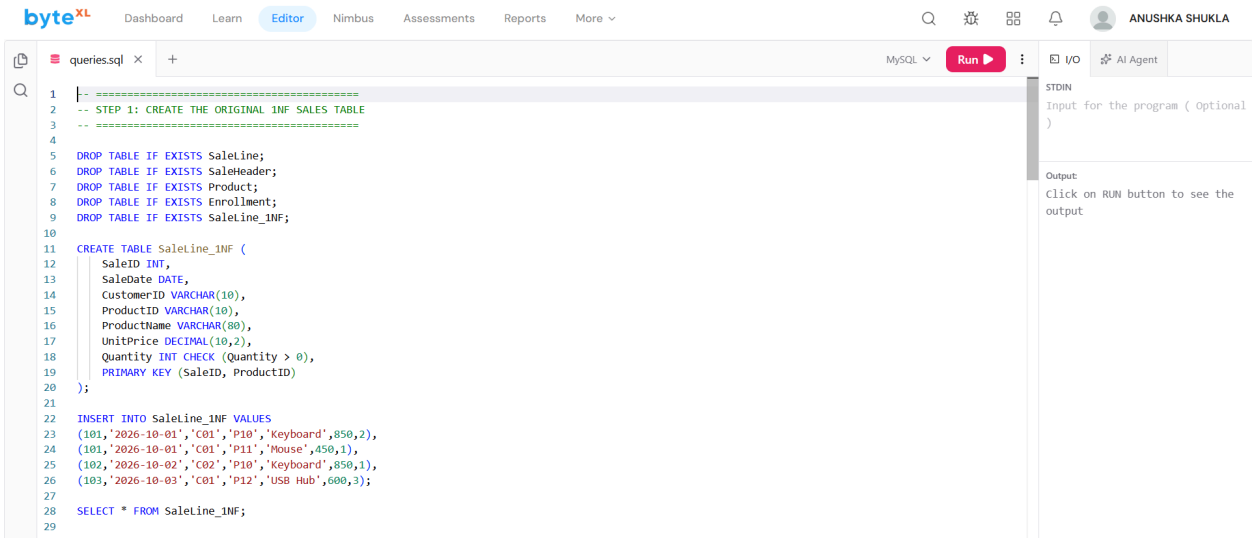
Lossless Join

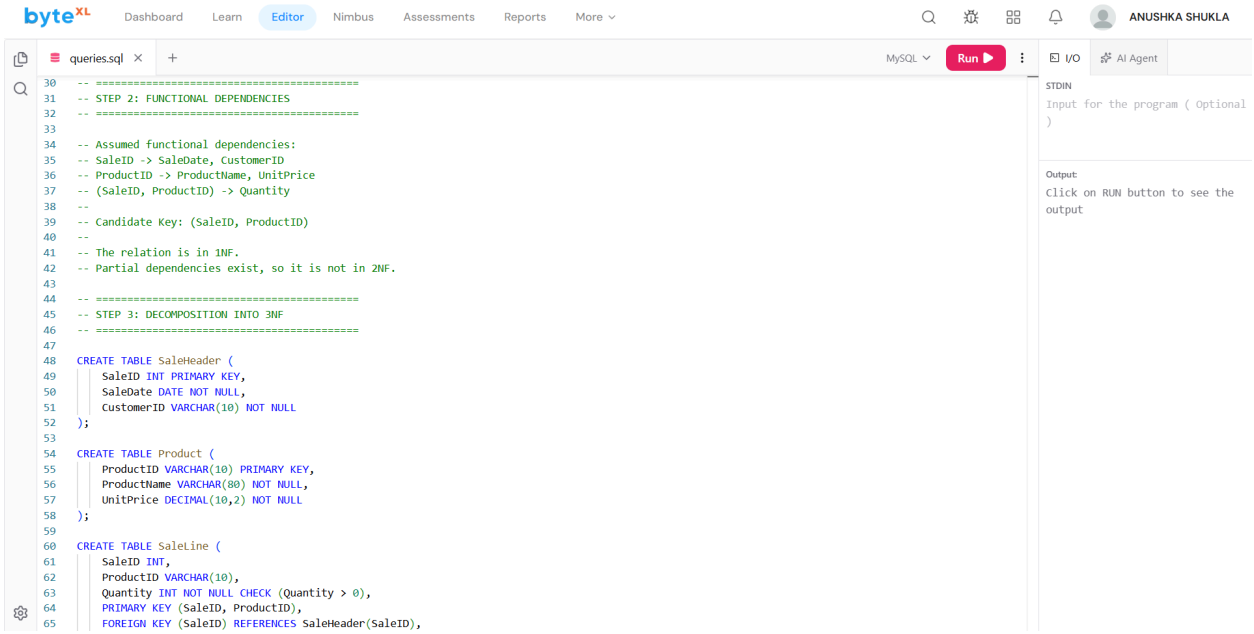
A decomposition is lossless if joining the decomposed relations reconstructs exactly the original relation without introducing spurious tuples.

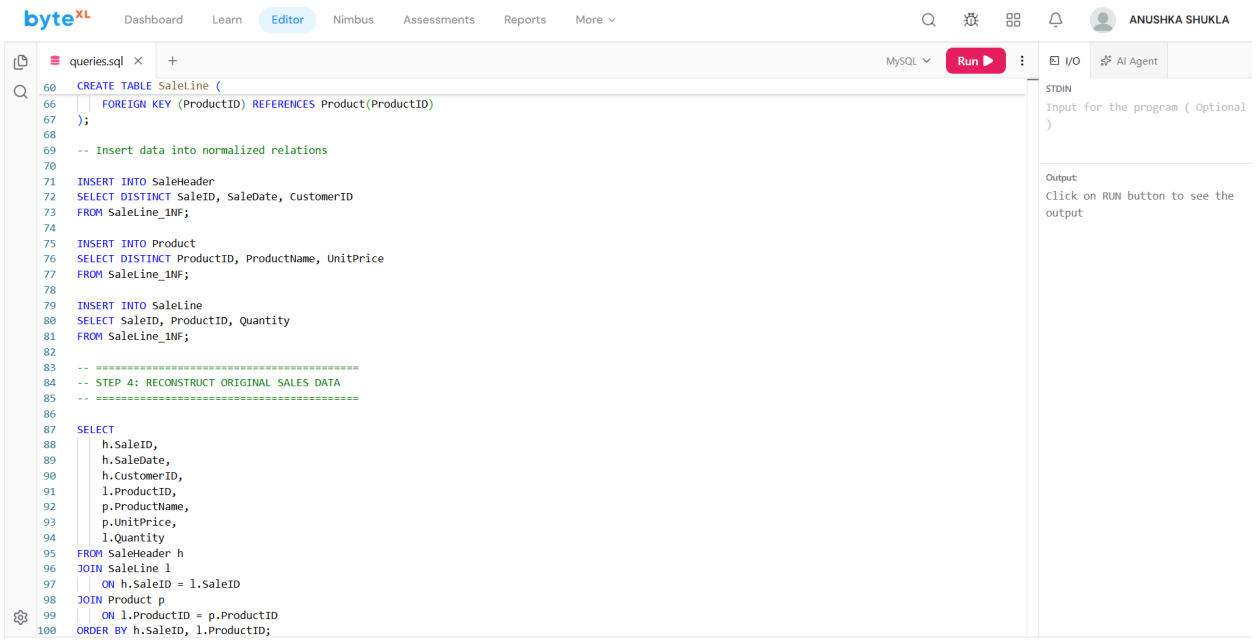
Dependency Preservation

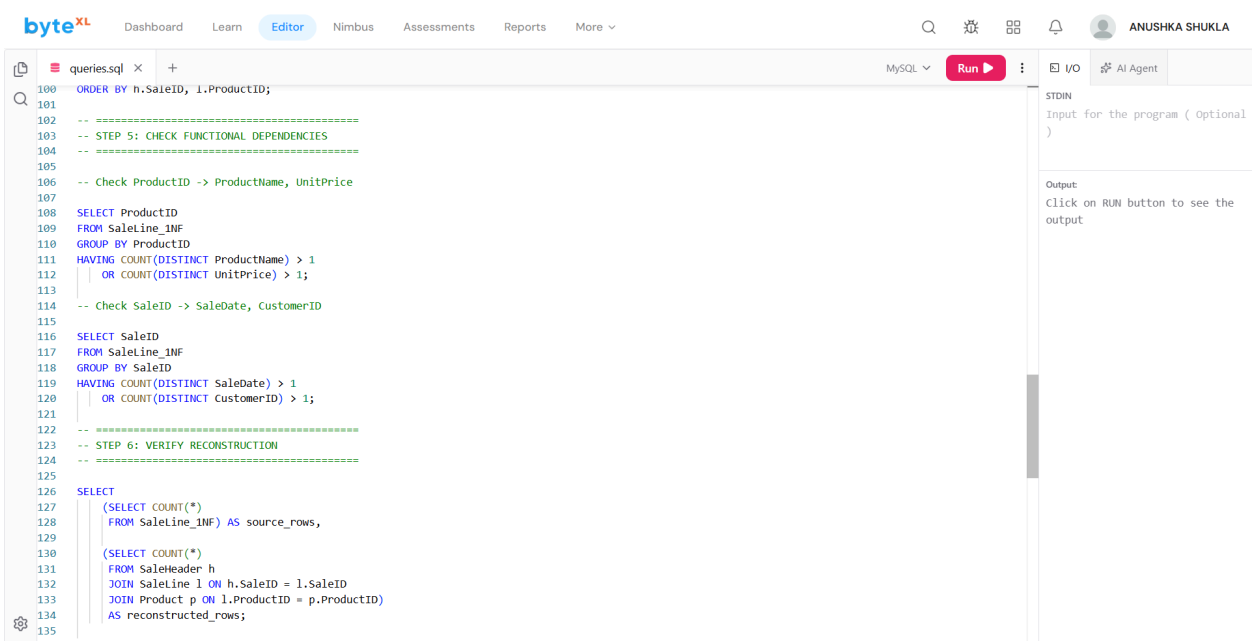
A decomposition is dependency-preserving if all functional dependencies can be enforced by checking the decomposed relations individually, without requiring joins.

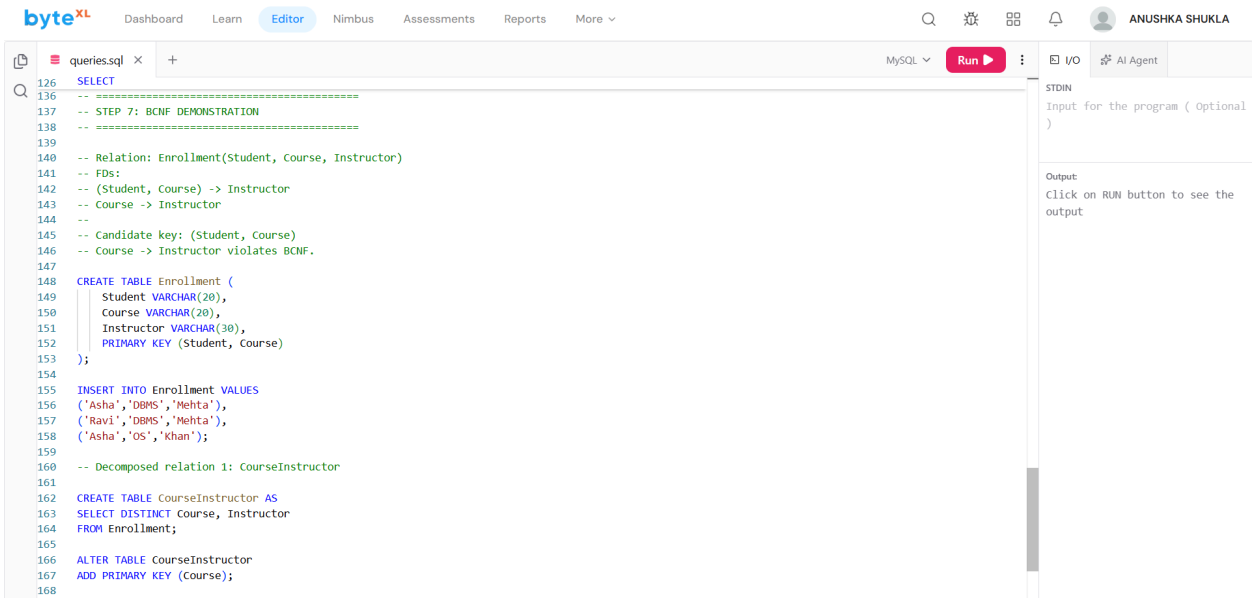
SOURCE CODE

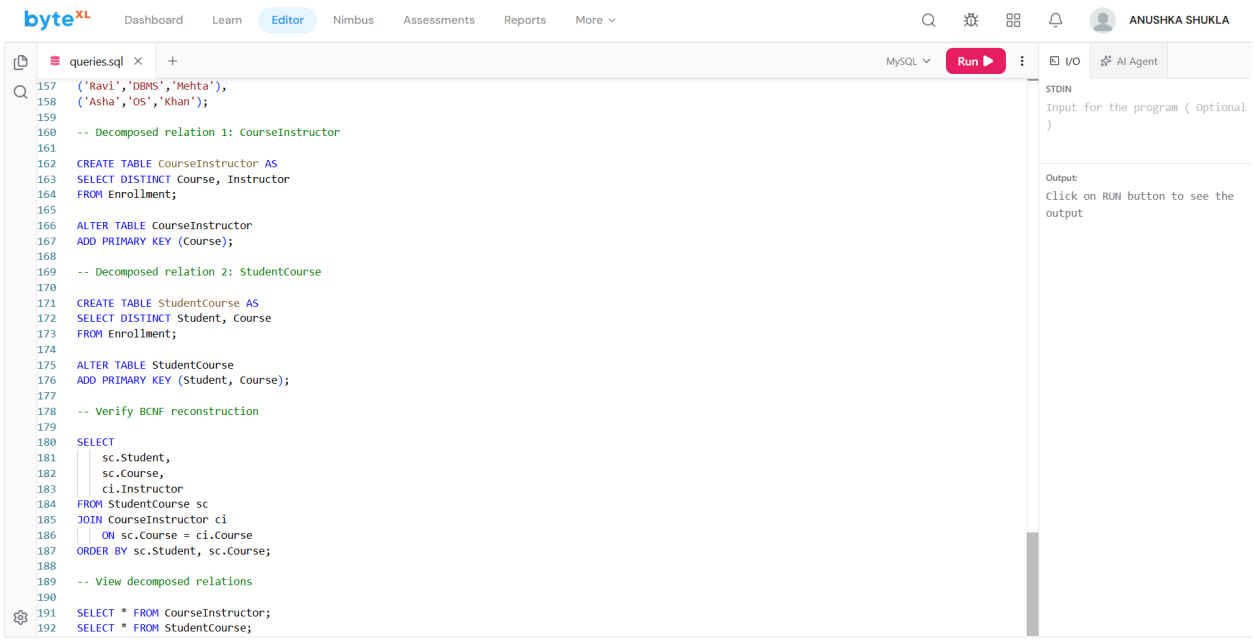

byteXL
Dashboard
Learn
Editor
Nimbus
Assessments
Reports
More
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
-- ==========
-- STEP 1: CREATE THE ORIGINAL 1NF SALES TABLE
-- ==========
DROP TABLE IF EXISTS SaleLine;
DROP TABLE IF EXISTS SaleHeader;
DROP TABLE IF EXISTS Product;
DROP TABLE IF EXISTS Enrollment;
DROP TABLE IF EXISTS SaleLine_1NF;
CREATE TABLE SaleLine_1NF (
SaleID INT,
SaleDate DATE,
CustomerID VARCHAR(10),
ProductID VARCHAR(10),
ProductName VARCHAR(80),
UnitPrice DECIMAL(10,2),
Quantity INT CHECK (Quantity > 0),
PRIMARY KEY (SaleID, ProductID)
);
INSERT INTO SaleLine_1NF VALUES
(101,'2026-10-01','C01','P10','Keyboard',850,2),
(101,'2026-10-01','C01','P11','Mouse',450,1),
(102,'2026-10-02','C02','P10','Keyboard',850,1),
(103,'2026-10-03','C01','P12','USB Hub',600,3);
SELECT * FROM SaleLine_1NF;


byteXL
Dashboard
Learn
Editor
Nimbus
Assessments
Reports
More
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
-- ==========
-- STEP 2: FUNCTIONAL DEPENDENCIES
-- ==========
-- Assumed functional dependencies:
-- SaleID -> SaleDate, CustomerID
-- ProductID -> ProductName, UnitPrice
-- (SaleID, ProductID) -> Quantity
--
-- Candidate Key: (SaleID, ProductID)
--
-- The relation is in 1NF.
-- Partial dependencies exist, so it is not in 2NF.
-- ==========
-- STEP 3: DECOMPOSITION INTO 3NF
-- ==========
CREATE TABLE SaleHeader (
SaleID INT PRIMARY KEY,
SaleDate DATE NOT NULL,
CustomerID VARCHAR(10) NOT NULL
);
CREATE TABLE Product (
ProductID VARCHAR(10) PRIMARY KEY,
ProductName VARCHAR(80) NOT NULL,
UnitPrice DECIMAL(10,2) NOT NULL
);
CREATE TABLE SaleLine (
SaleID INT,
ProductID VARCHAR(10),
Quantity INT NOT NULL CHECK (Quantity > 0),
PRIMARY KEY (SaleID, ProductID),
FOREIGN KEY (SaleID) REFERENCES SaleHeader(SaleID),

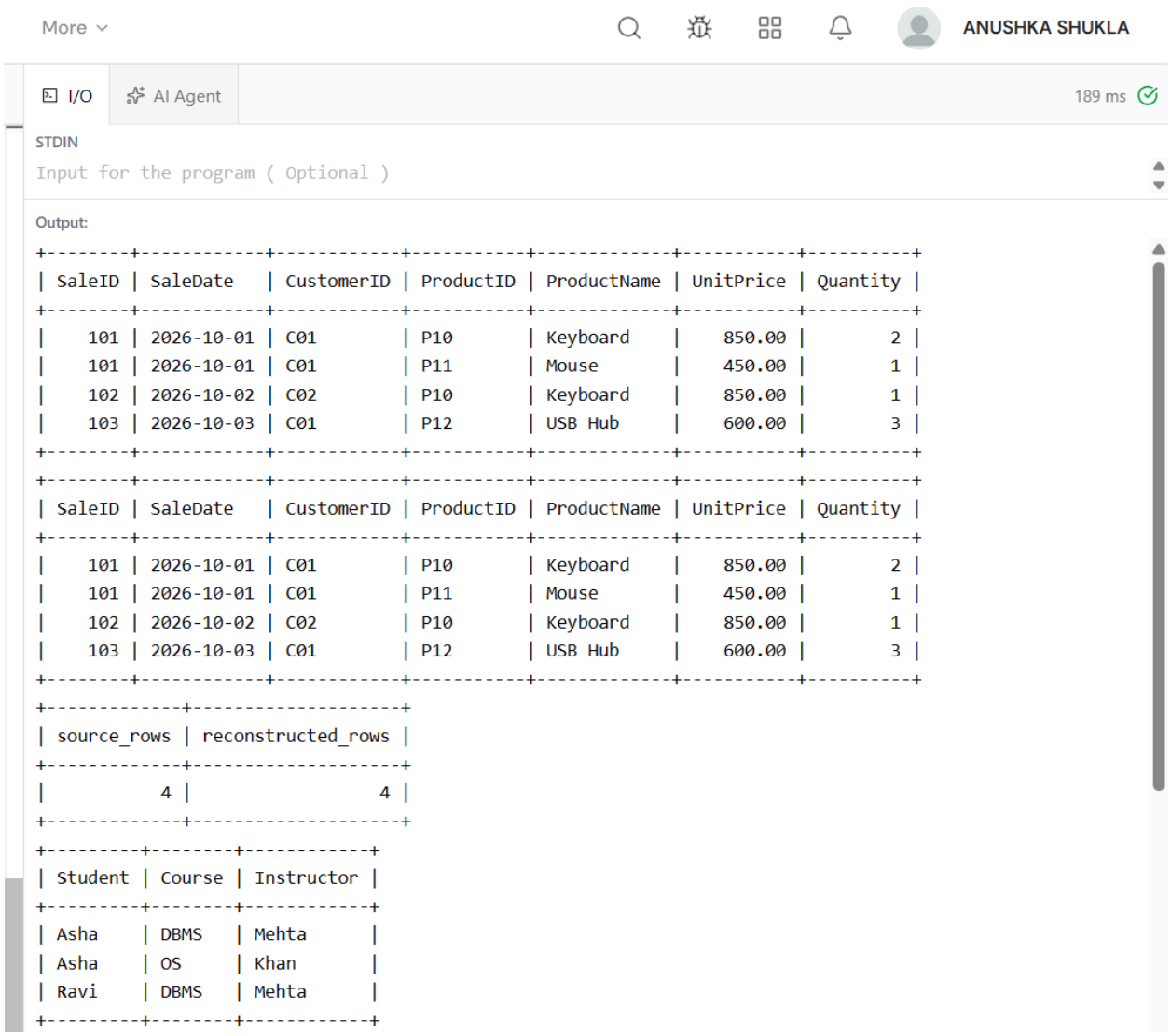

byteXL
Dashboard
Learn
Editor
Nimbus
Assessments
Reports
More
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
CREATE TABLE SaleLine (
FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
-- Insert data into normalized relations
INSERT INTO SaleHeader
SELECT DISTINCT SaleID, SaleDate, CustomerID
FROM SaleLine_1NF;
INSERT INTO Product
SELECT DISTINCT ProductID, ProductName, UnitPrice
FROM SaleLine_1NF;
INSERT INTO SaleLine
SELECT SaleID, ProductID, Quantity
FROM SaleLine_1NF;
-- ==
-- STEP 4: RECONSTRUCT ORIGINAL SALES DATA
-- ==
SELECT
h.SaleID,
h.SaleDate,
h.CustomerID,
l.ProductID,
p.ProductName,
p.UnitPrice,
l.Quantity
FROM SaleHeader h
JOIN SaleLine l
ON h.SaleID = l.SaleID
JOIN Product p
ON l.ProductID = p.ProductID
ORDER BY h.SaleID, l.ProductID;

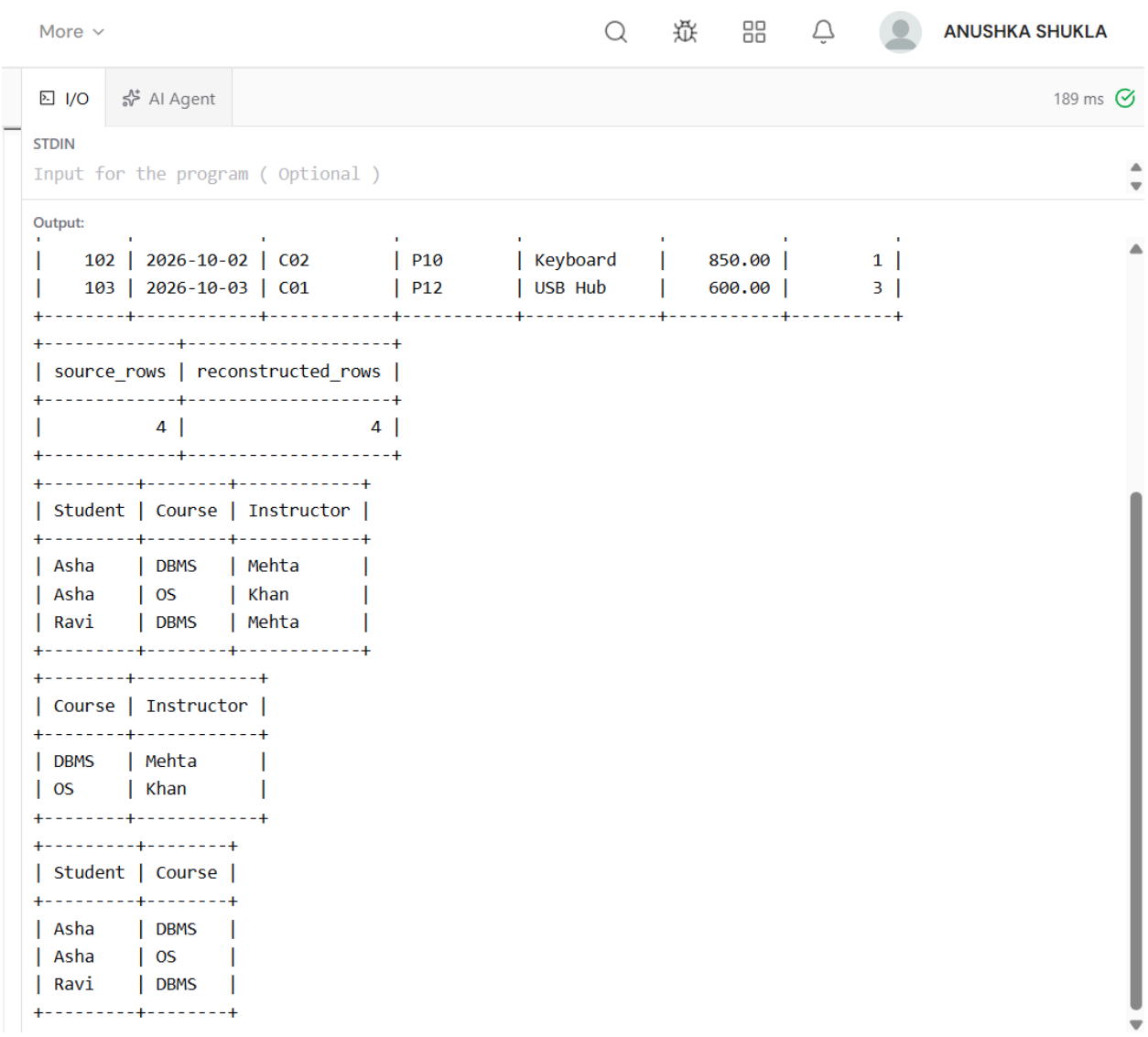

byteXL
Dashboard
Learn
Editor
Nimbus
Assessments
Reports
More
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
ORDER BY h.SaleID, l.ProductID;
-- ==
-- STEP 5: CHECK FUNCTIONAL DEPENDENCIES
-- ==
-- Check ProductID -> ProductName, UnitPrice
SELECT ProductID
FROM SaleLine_1NF
GROUP BY ProductID
HAVING COUNT(DISTINCT ProductName) > 1
OR COUNT(DISTINCT UnitPrice) > 1;
-- Check SaleID -> SaleDate, CustomerID
SELECT SaleID
FROM SaleLine_1NF
GROUP BY SaleID
HAVING COUNT(DISTINCT SaleDate) > 1
OR COUNT(DISTINCT CustomerID) > 1;
-- ==
-- STEP 6: VERIFY RECONSTRUCTION
-- ==
SELECT
(SELECT COUNT(*)
FROM SaleLine_1NF) AS source_rows,
(SELECT COUNT(*)
FROM SaleHeader h
JOIN SaleLine l ON h.SaleID = l.SaleID
JOIN Product p ON l.ProductID = p.ProductID)
AS reconstructed_rows;


byteXL
Dashboard
Learn
Editor
Nimbus
Assessments
Reports
More
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output
Click on RUN button to see the output
SELECT
-- ==
-- STEP 7: BCNF DEMONSTRATION
-- ==
-- Relation: Enrollment(Student, Course, Instructor)
-- FDs:
-- (Student, Course) -> Instructor
-- Course -> Instructor
--
-- Candidate key: (Student, Course)
-- Course -> Instructor violates BCNF.
CREATE TABLE Enrollment (
Student VARCHAR(20),
Course VARCHAR(20),
Instructor VARCHAR(30),
PRIMARY KEY (Student, Course)
);
INSERT INTO Enrollment VALUES
('Asha','DBMS','Mehta'),
('Ravi','DBMS','Mehta'),
('Asha','OS','Khan');
-- Decomposed relation 1: CourseInstructor
CREATE TABLE CourseInstructor AS
SELECT DISTINCT Course, Instructor
FROM Enrollment;
ALTER TABLE CourseInstructor
ADD PRIMARY KEY (Course);


byteXL
Dashboard
Learn
Editor
Nimbus
Assessments
Reports
More
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output
Click on RUN button to see the output
('Ravi','DBMS','Mehta'),
('Asha','OS','Khan');
-- Decomposed relation 1: CourseInstructor
CREATE TABLE CourseInstructor AS
SELECT DISTINCT Course, Instructor
FROM Enrollment;
ALTER TABLE CourseInstructor
ADD PRIMARY KEY (Course);
-- Decomposed relation 2: StudentCourse
CREATE TABLE StudentCourse AS
SELECT DISTINCT Student, Course
FROM Enrollment;
ALTER TABLE StudentCourse
ADD PRIMARY KEY (Student, Course);
-- Verify BCNF reconstruction
SELECT
sc.Student,
sc.Course,
ci.Instructor
FROM StudentCourse sc
JOIN CourseInstructor ci
ON sc.Course = ci.Course
ORDER BY sc.Student, sc.Course;
-- View decomposed relations
SELECT * FROM CourseInstructor;
SELECT * FROM StudentCourse;

OUTPUT


More
ANUSHKA SHUKLA
I/O
AI Agent
189 ms
STDIN
Input for the program (Optional)
Output:
+--------+------------+------------+-----------+-------------+-----------+----------+
| SaleID | SaleDate | CustomerID | ProductID | ProductName | UnitPrice | Quantity |
+--------+------------+------------+-----------+-------------+-----------+----------+
101	2026-10-01	C01	P10	Keyboard	850.00	2
101	2026-10-01	C01	P11	Mouse	450.00	1
102	2026-10-02	C02	P10	Keyboard	850.00	1
103	2026-10-03	C01	P12	USB Hub	600.00	3
+--------+------------+------------+-----------+-------------+-----------+----------+
+--------+------------+------------+-----------+-------------+-----------+----------+
| SaleID | SaleDate | CustomerID | ProductID | ProductName | UnitPrice | Quantity |
+--------+------------+------------+-----------+-------------+-----------+----------+
101	2026-10-01	C01	P10	Keyboard	850.00	2
101	2026-10-01	C01	P11	Mouse	450.00	1
102	2026-10-02	C02	P10	Keyboard	850.00	1
103	2026-10-03	C01	P12	USB Hub	600.00	3
+--------+------------+------------+-----------+-------------+-----------+----------+
+-------------+--------------------+
| source_rows | reconstructed_rows |
+-------------+--------------------+
| 4 | 4 |
+-------------+--------------------+
+---------+--------+------------+
| Student | Course | Instructor |
+---------+--------+------------+
Asha	DBMS	Mehta
Asha	OS	Khan
Ravi	DBMS	Mehta
+---------+--------+------------+



OBSERVATIONS/OUTCOME

1. The Sales relation contains repeated sale and product information.
2. The candidate key for the 1NF relation is (SaleID, ProductID).
3. SaleDate and CustomerID depend only on SaleID, whereas ProductName and UnitPrice depend only on ProductID.
4. The decomposition into SaleHeader, Product and SaleLine removes the identified partial dependencies.
5. The normalized relations can be joined to reconstruct the original sales information.
6. The dependency checks identify possible violations in the sample data.
7. In the Enrollment example, Course → Instructor violates BCNF.
8. The BCNF decomposition is lossless under the assumed functional dependency. General dependency preservation must be evaluated separately.